

# 从 `execve` 到新入口：Linux 如何装载并启动 ELF 程序

课程：《Linux内核原理与分析》

\*\*实验： \*\*实验七（第八周）

\*\*姓名： \*\*叶丁再

\*\*学号： \*\*20262809

\*\*日期： \*\*2026 年 9 月 27 日

## 一、问题：执行新程序时，旧程序去了哪里？

在 shell 或程序中执行一个命令，看起来像“启动了另一个程序”。Linux 的关键动作是 `execve`：内核检查并装载新的可执行文件，用新的地址空间、代码、数据和初始用户栈替换调用进程原来的用户态映像。成功时，`execve` 不会返回到旧程序的下一条指令；失败时才返回错误。

本实验围绕三个问题展开：ELF 文件怎样描述装载需求？`execve` 内核路径如何建立新程序的入口和栈？静态链接与动态链接程序开始运行时有什么不同？

## 二、实验环境与证据

实验在实验楼环境中完成，内核调试对象为 Linux 3.18.6 的 `vmlinux`，QEMU 通过 `bzImage` 和 `rootfs.img` 启动 MenuOS。用户态测试覆盖了静态、动态 ELF，装载时动态链接、运行时 `dlopen`，以及通过 `execv` 和 MenuOS 菜单执行程序。

下文将截图直接显示的内容称为“实验观察”；由 Linux 3.18.6 源码与 ELF 规则得出的内容称为“机制分析”。GDB 在 `sys_execve` 的栈回溯曾出现无法继续展开的提示，因此不把用户态调用栈说成已经完整动态跟踪。

## 三、先看 ELF：入口地址和解释器信息

ELF 文件包含 ELF header、程序头表、节区等信息。装载器主要依据程序头表中的 `PT_LOAD` 将文件内容映射到进程地址空间。ELF header 中的 `e_entry` 给出该 ELF 映像的入口地址；若存在 `PT_INTERP`，它还指出运行该映像所需的 ELF 解释器。

实验中的 `hello-static` 是 32 位 i386 静态 ELF，入口为 `0x8048d0a`，没有 `PT_INTERP`。`hello-dynamic` 同样是 32 位 i386 ELF，应用入口为 `0x8048350`，并带有 `/lib/ld-linux.so.2` 解释器段，动态节还声明依赖 `libc.so.6`。

Linux内核如何装载和启动一个可执行程序

Linux 内核如何装载和启动一个可执行程序

理解编译链接的过程和 ELF 可执行文件格式，详细内容参考本周第一节；

编程使用 execve 库函数加载一个可执行文件，动态链接分为可执行程序装载时动态链接和运行时动态链接，编程练习动态链接库的这两种使用方式，详细内容参考本周第二节；

使用 gdb 跟踪分析一个 execve 系统调用内核处理函数 sys\_execve，验证您对 Linux 系统加载可执行程序所需处理过程的理解，详细内容参考本周第三节；推荐在实验楼 Linux 虚拟机环境下完成实验。

特别关注新的可执行程序是从哪里开始执行的？为什么 execve 系统调用返回后新的可执行程序能顺利执行？对于静态链接的可执行程序 and 动态链接的可执行程序 execve 系统调用返回时会有什么不同？

根据本周所学知识分析 execve 函数对应的系统调用处理过程，撰写一篇署名博客，并在博客文章中注明“真实姓名（与最后申请证书的姓名务必一致）+ 原创作品转载请注明出处 + 《Linux 内核分析》MOOC 课程 <http://mooc.study.163.com/course/USTC-1000029000>”，博客内容的具体要求如下：

- 题目自拟，内容围绕对 Linux 内核如何装载和启动一个可执行程序；
- 可以结合实验截图、ELF 可执行文件格式、用户态的相关代码等；
- 博客内容中需要仔细分析新可执行程序的执行起点及对应的堆栈状态等。
- 总结部分需要阐明自己对“Linux 内核装载和启动一个可执行程序”的理解

请提交博客文章 URL 到网易云课堂 MOOC 平台，编辑成一个链接可以直接点击打开。（学堂在线上的学员注意提交博客 URL 到学堂在线上）。

完成学习

shiyanolou@gab857f96f8c024125cd9bd5: ~/LinuxKernel/exec-lab

```
readelf: 错误: 'hello-dynamic': No such file
shiyanolou@exec-lab/ $ cd ~/LinuxKernel/exec-lab
shiyanolou@exec-lab/ $ ls -ls src bin
bin
总用量 732
-rwxrwxr-x 1 shiyanolou shiyanolou 7330 9月 27 07:46 hello-dynamic
-rw-rw-r-- 1 shiyanolou shiyanolou 1212 9月 27 07:46 hello.o
-rwxrwxr-x 1 shiyanolou shiyanolou 73329 9月 27 07:46 hello-static
src:
总用量 28
-rw-rw-r-- 1 shiyanolou shiyanolou 242 9月 27 07:46 hello.c
-rw-rw-r-- 1 shiyanolou shiyanolou 17491 9月 27 07:46 hello.l
-rwxrwxr-x 1 shiyanolou shiyanolou 949 9月 27 07:46 hello.s
shiyanolou@exec-lab/ $ file bin/hello-dynamic
bin/hello-dynamic: ELF 32-bit LSB executable, Intel 80386, version 1 (GNU/Linux), statically linked, for GNU/Linux 2.6.24, BuildID[sha1]=d2c1ac215fc55ca92da828bd34dcc834b6d0c473, no stripped
bin/hello-dynamic: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked (uses shared libs), for GNU/Linux 2.6.24, BuildID[sha1]=db7bde88f8e1e7725749c3c4880ba8f7ce10489, not stripped
shiyanolou@exec-lab/ $ readelf -h bin/hello-static
ELF 头:
Magic: 7f 45 4c 46 01 01 01 03 00 00 00 00 00 00 00 00
Class: ELF32
Data: 2's complement, little endian
Version: 1 (current)
OS/ABI: UNIX - GNU
ABI Version: 0
Type: EXEC (可执行文件)
Machine: Intel 80386
Version: 0x1
人口点地址: 0x804800a
程序头起点: 52 (bytes into file)
Start of section headers: 663792 (bytes into file)
标志: 0x0
本头的大小: 52 (字节)
程序头大小: 32 (字节)
Number of program headers: 6
节头大小: 40 (字节)
节头数量: 31
字符串表索引节头: 28
shiyanolou@exec-lab/ $ readelf -l bin/hello-static
ELF 文件类型为 EXEC (可执行文件)
人口点 0x804800a
共有 6 个程序头，开始于偏移量 52

应用程序菜单
```

Linux内核如何装载和启动一个可执行程序

Linux 内核如何装载和启动一个可执行程序

理解编译链接的过程和 ELF 可执行文件格式，详细内容参考本周第一节；

编程使用 execve 库函数加载一个可执行文件，动态链接分为可执行程序装载时动态链接和运行时动态链接，编程练习动态链接库的这两种使用方式，详细内容参考本周第二节；

使用 gdb 跟踪分析一个 execve 系统调用内核处理函数 sys\_execve，验证您对 Linux 系统加载可执行程序所需处理过程的理解，详细内容参考本周第三节；推荐在实验楼 Linux 虚拟机环境下完成实验。

特别关注新的可执行程序是从哪里开始执行的？为什么 execve 系统调用返回后新的可执行程序能顺利执行？对于静态链接的可执行程序 and 动态链接的可执行程序 execve 系统调用返回时会有什么不同？

根据本周所学知识分析 execve 函数对应的系统调用处理过程，撰写一篇署名博客，并在博客文章中注明“真实姓名（与最后申请证书的姓名务必一致）+ 原创作品转载请注明出处 + 《Linux 内核分析》MOOC 课程 <http://mooc.study.163.com/course/USTC-1000029000>”，博客内容的具体要求如下：

- 题目自拟，内容围绕对 Linux 内核如何装载和启动一个可执行程序；
- 可以结合实验截图、ELF 可执行文件格式、用户态的相关代码等；
- 博客内容中需要仔细分析新可执行程序的执行起点及对应的堆栈状态等。
- 总结部分需要阐明自己对“Linux 内核装载和启动一个可执行程序”的理解

请提交博客文章 URL 到网易云课堂 MOOC 平台，编辑成一个链接可以直接点击打开。（学堂在线上的学员注意提交博客 URL 到学堂在线上）。

完成学习

shiyanolou@gab857f96f8c024125cd9bd5: ~/LinuxKernel/exec-lab

```
Machine: Intel 80386
Version: 0x1
人口点地址: 0x8048350
程序头起点: 52 (bytes into file)
Start of section headers: 4432 (bytes into file)
标志: 0x0
本头的大小: 52 (字节)
程序头大小: 32 (字节)
Number of program headers: 9
节头大小: 40 (字节)
节头数量: 30
字符串表索引节头: 27
shiyanolou@exec-lab/ $ readelf -l bin/hello-dynamic
ELF 文件类型为 EXEC (可执行文件)
人口点 0x8048350
共有 9 个程序头，开始于偏移量 52

程序头:
Type Offset VirtAddr PhysAddr FileSiz MemSiz Flg Align
PHDR 0x000034 0x00048034 0x00048034 0x00120 0x00120 R E 0x4
INTERP 0x000154 0x00048154 0x00048154 0x00013 0x00013 R 0x1
LOAD [正在请求程序解释器: /lib/ld-linux.so.2]
LOAD 0x000000 0x00048000 0x00048000 0x00668 0x00668 R E 0x1000
LOAD 0x000f08 0x00049f08 0x00049f08 0x0011c 0x00120 RW 0x1000
DYNAMIC 0x000f14 0x00049f14 0x00049f14 0x000e8 0x000e8 RW 0x4
NOTE 0x000168 0x00048168 0x00048168 0x00044 0x00044 R 0x4
GNU_EH_FRAME 0x00058c 0x0004858c 0x0004858c 0x0002c 0x0002c R 0x4
GNU_STACK 0x000000 0x00000000 0x00000000 0x00000 0x00000 RW 0x10
GNU_RELRO 0x000f08 0x00049f08 0x00049f08 0x000f8 0x000f8 R 0x1

Section to Segment mapping:
段节...
00
01 .interp
02 .interp.note.ABI-tag.note.gnu.build-id.gnu.hash.dynsym.dynstr.gnu.version.gnu.version_r.rel.dyn.rel.plt.init.plt.text.fini.rodata.eh_frame_hdr.eh_frame
03 .init_array.fini_array.jcr.dynamic.got.got.plt.data.bss
04 .dynamic
05 .note.ABI-tag.note.gnu.build-id
06 .eh_frame_hdr
07
08 .init_array.fini_array.jcr.dynamic.got
shiyanolou@exec-lab/ $ readelf -d bin/hello-dynamic

应用程序菜单
```

两种动态库使用方式

实验还用 libgreet.so 对比了两类动态链接。greet-linked 的动态节中有 NEEDED: libgreet.so，并通过 \$ORIGIN/../lib 查找库；程序启动装载时，动态链接器会解析这个依赖。greet-dlopen 的动态节没有 libgreet.so 这一项，而是由程序运行到相应代码后调用 dlopen 打开库，再通过 dlsym 找到 greet 函数。两种程序都实际输出了问候信息。

2 / 15

Linux内核如何装载和启动一个可执行程序

Linux 内核如何装载和启动一个可执行程序

理解编译链接的过程和 ELF 可执行文件格式。详细内容参考本周第一节；

编程使用 exec\*库函数加载一个可执行文件，动态链接分为可执行程序装载时动态链接和运行时动态链接，编程练习动态链接库的这两种使用方式，详细内容参考本周第二节；

使用 gdb 跟踪分析一个 execve 系统调用内核处理函数 sys\_execve，验证您对 Linux 系统加载可执行程序所需处理过程的理解。详细内容参考本周第三节；推荐在实验楼 Linux 虚拟机环境下完成实验。

特别关注新的可执行程序是从哪里开始执行的？为什么 execve 系统调用返回后新的可执行程序能顺利执行？对于静态链接的可执行程序 and 动态链接的可执行程序 execve 系统调用返回时会有什么不同？

根据本周所学知识分析 exec\* 函数对应的系统调用处理过程，撰写一篇署名博客，并在博客文章中注明“真实姓名（与最后申请证书的姓名务必一致）+ 原创作品转载请注明出处 + 《Linux 内核分析》MOOC 课程 <http://moooc.study163.com/course/USTC-1000029000/>”，博客内容的具体要求如下：

题目自拟，内容围绕对 Linux 内核如何装载和启动一个可执行程序；

可以结合实验截图、ELF 可执行文件格式、用户态的相关代码等；

博客内容中需要仔细分析新可执行程序的执行起点及对应的堆栈状态等。

总结部分需要阐明自己对“Linux 内核装载和启动一个可执行程序”的理解

请提交博客文章 URL 到网易云课堂 MOOC 平台，编辑成一个链接可以直接点击打开。（学堂在线上的学员注意提交博客 URL 到学堂在线上）。

完成学习

shiyanolou@6ab857f96f8c024125cd9bd5: ~/LinuxKernel/exec-lab

```
heredoc> const char *error;
heredoc> greet_fn greet;
heredoc>
heredoc> if (handle == NULL) {
heredoc>     fprintf(stderr, "dlopen: %s\n", dlerror());
heredoc>     return 1;
heredoc> }
heredoc>
heredoc> dlerror();
heredoc> greet = (greet_fn)dlsym(handle, "greet");
heredoc> error = dlerror();
heredoc> if (error != NULL) {
heredoc>     fprintf(stderr, "dlsym: %s\n", error);
heredoc>     dlclose(handle);
heredoc>     return 1;
heredoc> }
heredoc>
heredoc> greet("run-time dlopen");
heredoc> dlclose(handle);
heredoc> return 0;
heredoc> }
heredoc> EOF
shiyanolou:exec-lab/ $ gcc -m32 -fPIC -shared src/greet.c -o lib/libgreet.so [8:01:00]
shiyanolou:exec-lab/ $ gcc -m32 src/greet-linked.c -llib -Wl,-rpath,'$ORIGIN/../lib' -lgreet -o bin/greet-linked
reel-linked
shiyanolou:exec-lab/ $ gcc -m32 src/greet-dlopen.c -o bin/greet-dlopen -ldl [8:02:26]
shiyanolou:exec-lab/ $ [8:02:26]
shiyanolou:exec-lab/ $ ./bin/greet-linked [8:02:26]
Calling a library linked at program load time:
Hello, load-time linking from libgreet.so
shiyanolou:exec-lab/ $ ./bin/greet-dlopen [8:02:26]
Hello, run-time dlopen from libgreet.so
shiyanolou:exec-lab/ $ █ [8:02:28]
```

应用程序菜单

Linux内核如何装载和启动一个可执行程序

Linux 内核如何装载和启动一个可执行程序

理解编译链接的过程和 ELF 可执行文件格式。详细内容参考本周第一节；

编程使用 exec\*库函数加载一个可执行文件，动态链接分为可执行程序装载时动态链接和运行时动态链接，编程练习动态链接库的这两种使用方式，详细内容参考本周第二节；

使用 gdb 跟踪分析一个 execve 系统调用内核处理函数 sys\_execve，验证您对 Linux 系统加载可执行程序所需处理过程的理解。详细内容参考本周第三节；推荐在实验楼 Linux 虚拟机环境下完成实验。

特别关注新的可执行程序是从哪里开始执行的？为什么 execve 系统调用返回后新的可执行程序能顺利执行？对于静态链接的可执行程序 and 动态链接的可执行程序 execve 系统调用返回时会有什么不同？

根据本周所学知识分析 exec\* 函数对应的系统调用处理过程，撰写一篇署名博客，并在博客文章中注明“真实姓名（与最后申请证书的姓名务必一致）+ 原创作品转载请注明出处 + 《Linux 内核分析》MOOC 课程 <http://moooc.study163.com/course/USTC-1000029000/>”，博客内容的具体要求如下：

题目自拟，内容围绕对 Linux 内核如何装载和启动一个可执行程序；

可以结合实验截图、ELF 可执行文件格式、用户态的相关代码等；

博客内容中需要仔细分析新可执行程序的执行起点及对应的堆栈状态等。

总结部分需要阐明自己对“Linux 内核装载和启动一个可执行程序”的理解

请提交博客文章 URL 到网易云课堂 MOOC 平台，编辑成一个链接可以直接点击打开。（学堂在线上的学员注意提交博客 URL 到学堂在线上）。

完成学习

shiyanolou@6ab857f96f8c024125cd9bd5: ~/LinuxKernel/exec-lab

```
Hello, run-time dlopen from libgreet.so
shiyanolou:exec-lab/ $ readelf -d bin/greet-linked [8:02:28]

Dynamic section at offset 0xf04 contains 26 entries:
 标记      类型      名称/值
0x00000001 (NEEDED)      共享库: [libgreet.so]
0x00000001 (NEEDED)      共享库: [libc.so.6]
0x0000000f (RPATH)      Library rpath: [$ORIGIN/../lib]
0x0000000c (INIT)      0x8048408
0x0000000d (FINI)      0x8048624
0x00000019 (INIT_ARRAY)  0x8049ef8
0x0000001b (INIT_ARRAYSZ) 4 (bytes)
0x0000001a (FINI_ARRAY)  0x8049efc
0x0000001c (FINI_ARRAYSZ) 4 (bytes)
0x6ffffff5 (GNU_HASH)    0x80481ac
0x00000005 (STRTAB)      0x80482c8
0x00000006 (SYMTAB)      0x80481e8
0x0000000a (STRSZ)      217 (bytes)
0x0000000b (SYMENT)      16 (bytes)
0x00000015 (DEBUG)      0x0
0x00000003 (PLTGOT)      0x804a000
0x00000002 (PLTRELSZ)    32 (bytes)
0x00000014 (PLTREL)      REL
0x00000017 (JMPREL)      0x80483e8
0x00000011 (REL)         0x80483e0
0x00000012 (RELSZ)      8 (bytes)
0x00000013 (RELENT)      8 (bytes)
0x6ffffffe (VERNEED)     0x80483c0
0x6fffffff (VERNEEDNUM)  1
0x6ffffff0 (VERSYM)      0x80483a2
0x00000000 (NULL)        0x0
shiyanolou:exec-lab/ $ readelf -d bin/greet-dlopen [8:02:50]

Dynamic section at offset 0xf0c contains 25 entries:
 标记      类型      名称/值
0x00000001 (NEEDED)      共享库: [libdl.so.2]
0x00000001 (NEEDED)      共享库: [libc.so.6]
0x0000000c (INIT)      0x8048424
0x0000000d (FINI)      0x8048714
0x00000019 (INIT_ARRAY)  0x8049f00
0x0000001b (INIT_ARRAYSZ) 4 (bytes)
0x0000001a (FINI_ARRAY)  0x8049f04
0x0000001c (FINI_ARRAYSZ) 4 (bytes)
0x6ffffff5 (GNU_HASH)    0x80481ac
0x00000005 (STRTAB)      0x80482a0
0x00000006 (SYMTAB)      0x80481d0
0x0000000a (STRSZ)      208 (bytes)
0x0000000b (SYMENT)      16 (bytes)
0x00000015 (DEBUG)      0x0
0x00000003 (PLTGOT)      0x804a000
0x00000002 (PLTRELSZ)    56 (bytes)
0x00000014 (PLTREL)      REL
0x00000017 (JMPREL)      0x80483ec
0x00000011 (REL)         0x80483dc
0x00000012 (RELSZ)      16 (bytes)
0x00000013 (RELENT)      8 (bytes)
0x6ffffffe (VERNEED)     0x804838c
0x6fffffff (VERNEEDNUM)  2
0x6ffffff0 (VERSYM)      0x8048370
0x00000000 (NULL)        0x0
shiyanolou:exec-lab/ $ █ [8:02:53]
```

应用程序菜单

3 / 15

## 四、用户态 `execv` 测试

`execv(path, argv)` 是 `libc` 提供的执行接口之一，底层请求内核执行指定路径。下面是实验使用的执行驱动程序的核心逻辑（整理了截图中因终端编码显示异常的提示字符串；路径选择、参数构造和 `execv` 调用与实验一致）：

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    const char *path;
    char *child_argv[3];

    if (argc != 2 ||
        (strcmp(argv[1], "static") != 0 &&
         strcmp(argv[1], "dynamic") != 0)) {
        fprintf(stderr, "usage: %s static|dynamic\n", argv[0]);
        return 1;
    }

    path = (strcmp(argv[1], "static") == 0)
        ? "./bin/hello-static"
        : "./bin/hello-dynamic";

    child_argv[0] = (char *)path;
    child_argv[1] = "from-execv";
    child_argv[2] = NULL;

    printf("execv -> %s\n", path);
    fflush(stdout);
    execv(path, child_argv);

    /* 只有 execv 失败，控制流才会到达这里。 */
    perror("execv");
    return 1;
}
```

运行截图中，静态和动态目标程序都输出 `Hello from the new executable!`，并显示 `argc = 2`、`argv[0]` 为程序路径、`argv[1] = from-execv`。新程序读取到的参数说明内核在建立新用户栈时也带入了调用者提供的参数向量。

Linux内核如何装载和启动一个可执行程序

## Linux 内核如何装载和启动一个可执行程序

- 理解编译链接的过程和 ELF 可执行文件格式，详细内容参考本周第一节；
- 编程使用 `exec` 库函数加载一个可执行文件，动态链接分为可执行程序装载时动态链接和运行时动态链接，编程练习动态链接库的这两种使用方式，详细内容参考本周第二节；
- 使用 `gdb` 跟踪分析一个 `execve` 系统调用内核处理函数 `sys_execve`，验证您对 Linux 系统加载可执行程序所需处理过程的理解，详细内容参考本周第三节；推荐在实验楼 Linux 虚拟机环境下完成实验。
- 特别关注新的可执行程序是从哪里开始执行的？为什么 `execve` 系统调用返回后新的可执行程序能顺利执行？对于静态链接的可执行程序 and 动态链接的可执行程序 `execve` 系统调用返回时会有什么不同？
- 根据本周所学知识分析 `exec*` 函数对应的系统调用处理过程，撰写一篇署名博客，并在博客文章中注明“真实姓名（与最后申请证书的姓名务必一致）+ 原创作品转载请注明出处 + 《Linux 内核分析》MOOC 课程 <http://mooc.study.163.com/course/USTC-1000029000>”，博客内容的具体要求如下：
  - 题目自拟，内容围绕对 Linux 内核如何装载和启动一个可执行程序；
  - 可以结合实验截图、ELF 可执行文件格式、用户态的相关代码等；
  - 博客内容中需要仔细分析新可执行程序的执行起点及对应的堆栈状态等。
  - 总结部分需要阐明自己对“Linux 内核装载和启动一个可执行程序”的理解

请提交博客文章 URL 到网易云课堂 MOOC 平台，编辑成一个链接可以直接点击打开。（学堂在线上的学员注意提交博客 URL 到学堂在线上）。

完成学习

```
shiyanolou@6ab857f96f8c024125cd9bd5: ~/LinuxKernel/exec-lab
shiyanolou:exec-lab/ $ cat > src/exec_driver.c <<'EOF'
heredoc> #include <stdio.h>
heredoc> #include <string.h>
heredoc> #include <unistd.h>
heredoc>
heredoc> int main(int argc, char *argv[])
heredoc> {
heredoc>     const char *path;
heredoc>     char *child_argv[3];
heredoc>
heredoc>     if (argc != 2 ||
heredoc>         (strcmp(argv[1], "static") != 0 &&
heredoc>          strcmp(argv[1], "dynamic") != 0)) {
heredoc>         fprintf(stderr, "???%s static|dynamic\n", argv[0]);
heredoc>         return 1;
heredoc>     }
heredoc>
heredoc>     if (strcmp(argv[1], "static") == 0)
heredoc>         path = "./bin/hello-static";
heredoc>     else
heredoc>         path = "./bin/hello-dynamic";
heredoc>
heredoc>     child_argv[0] = (char *)path;
heredoc>     child_argv[1] = "from-execv";
heredoc>     child_argv[2] = NULL;
heredoc>
heredoc>     printf("execv ?????????? %s\n", path);
heredoc>     fflush(stdout);
heredoc>
heredoc>     execv(path, child_argv);
heredoc>
heredoc>     /* execv ?????????? */
heredoc>     perror("execv");
heredoc>     return 1;
heredoc> }
```

应用程序菜单

Linux内核如何装载和启动一个可执行程序

## Linux 内核如何装载和启动一个可执行程序

- 理解编译链接的过程和 ELF 可执行文件格式，详细内容参考本周第一节；
- 编程使用 `exec` 库函数加载一个可执行文件，动态链接分为可执行程序装载时动态链接和运行时动态链接，编程练习动态链接库的这两种使用方式，详细内容参考本周第二节；
- 使用 `gdb` 跟踪分析一个 `execve` 系统调用内核处理函数 `sys_execve`，验证您对 Linux 系统加载可执行程序所需处理过程的理解，详细内容参考本周第三节；推荐在实验楼 Linux 虚拟机环境下完成实验。
- 特别关注新的可执行程序是从哪里开始执行的？为什么 `execve` 系统调用返回后新的可执行程序能顺利执行？对于静态链接的可执行程序 and 动态链接的可执行程序 `execve` 系统调用返回时会有什么不同？
- 根据本周所学知识分析 `exec*` 函数对应的系统调用处理过程，撰写一篇署名博客，并在博客文章中注明“真实姓名（与最后申请证书的姓名务必一致）+ 原创作品转载请注明出处 + 《Linux 内核分析》MOOC 课程 <http://mooc.study.163.com/course/USTC-1000029000>”，博客内容的具体要求如下：
  - 题目自拟，内容围绕对 Linux 内核如何装载和启动一个可执行程序；
  - 可以结合实验截图、ELF 可执行文件格式、用户态的相关代码等；
  - 博客内容中需要仔细分析新可执行程序的执行起点及对应的堆栈状态等。
  - 总结部分需要阐明自己对“Linux 内核装载和启动一个可执行程序”的理解

请提交博客文章 URL 到网易云课堂 MOOC 平台，编辑成一个链接可以直接点击打开。（学堂在线上的学员注意提交博客 URL 到学堂在线上）。

完成学习

```
shiyanolou@6ab857f96f8c024125cd9bd5: ~/LinuxKernel/exec-lab
shiyanolou:exec-lab/ $ cd ~/LinuxKernel/exec-lab
shiyanolou:exec-lab/ $ gcc -m32 src/exec_driver.c -o bin/exec-driver
shiyanolou:exec-lab/ $ ./bin/exec-driver static
execv ?????????? ./bin/hello-static
Hello from the new executable!
argc = 2
argv[0] = ./bin/hello-static
argv[1] = from-execv
shiyanolou:exec-lab/ $ ./bin/exec-driver dynamic
execv ?????????? ./bin/hello-dynamic
Hello from the new executable!
argc = 2
argv[0] = ./bin/hello-dynamic
argv[1] = from-execv
shiyanolou:exec-lab/ $
```

应用程序菜单

MenuOS 测试采用 `fork` 后由子进程执行 `execv`，父进程等待子进程结束。QEMU 中分别执行静态和动态目标，二者都输出 `argv[1] = from-MenuOS`，子进程退出码为 0，MenuOS 提示符恢复。这同时验证了 `initramfs` 中程序及动态解释器、共享库等依赖可被找到。

Linux内核如何装载和启动一个可执行程序

## Linux 内核如何装载和启动一个可执行程序

- 理解编译链接的过程和 ELF 可执行文件格式，详细内容参考本周第一节；
- 编程使用 `exec` 库函数加载一个可执行文件，动态链接分为可执行程序装载时动态链接和运行时动态链接，编程练习动态链接库的这两种使用方式，详细内容参考本周第二节；
- 使用 `gdb` 跟踪分析一个 `execve` 系统调用内核处理函数 `sys_execve`，验证您对 Linux 系统加载可执行程序所需处理过程的理解，详细内容参考本周第三节；推荐在实验楼 Linux 虚拟机环境下完成实验。
- 特别关注新的可执行程序是从哪里开始执行的？为什么 `execve` 系统调用返回后新的可执行程序能顺利执行？对于静态链接的可执行程序 and 动态链接的可执行程序 `execve` 系统调用返回时会有什么不同？
- 根据本周所学知识分析 `exec*` 函数对应的系统调用处理过程，撰写一篇署名博客，并在博客文章中注明“真实姓名（与最后申请证书的姓名务必一致）+ 原创作品转载请注明出处 + 《Linux 内核分析》MOOC 课程 <http://moo.study163.com/course/USTC-1000029000/>”，博客内容的具体要求如下：
  - 题目自拟，内容围绕对 Linux 内核如何装载和启动一个可执行程序；
  - 可以结合实验截图、ELF 可执行文件格式、用户态的相关代码等；
  - 博客内容中需要仔细分析新可执行程序的执行起点及对应的堆栈状态等。
  - 总结部分需要阐明自己对“Linux 内核装载和启动一个可执行程序”的理解

请提交博客文章 URL 到网易云课堂 MOOC 平台，编辑成一个链接可以直接点击打开。（学堂在线上的学员注意提交博客 URL 到学堂在线上）。

完成学习

```
qemu -kernel ../linux-3.18.6/arch/x86/boot/bzImage -initrd ../rootfs.img
shiyancelou:~/ $ cd ~/LinuxKernel/menu
shiyancelou:menu/ (master*) $ qemu -kernel ../linux-3.18.6/arch/x86/boot/bzImage -
initrd ../rootfs.img

QEMU - Press Ctrl-Alt to exit mouse grab
[ 2.663886] Write protecting the kernel text: 7488k
[ 2.664297] Write protecting the kernel read-only data: 2444k
[ 2.695329] kworker/u2:1 (001) used greatest stack depth: 6996 bytes left

MenuOS>[ 3.064782] input: ImExPS/2 Generic Explorer Mouse as /devices/platform/
rm/18042/serial/input/input3
[ 3.179199] Switched to clocksource tsc

This is a wrong cmd!
MenuOS>exec
Hello from the new executable!
argc = 2
argv[0] = ./hello-static
argv[1] = from-MenuOS
exec child exit=0
MenuOS>
```

41:08 延时

切换界面

关闭环境

剪贴板

共享桌面

下载代码

上传文件

桌面全屏

环境信息

SSH直连

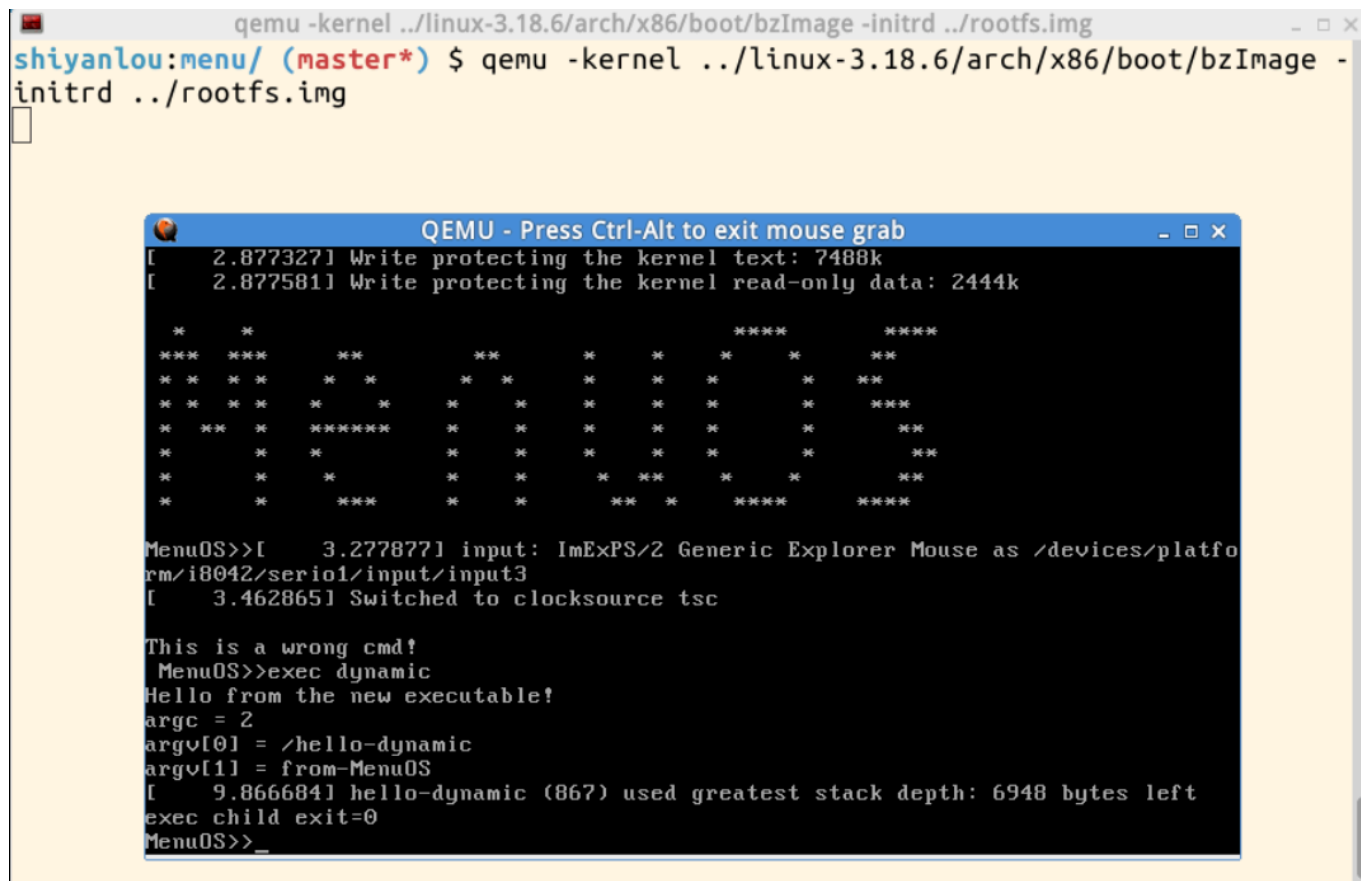
保存环境

更多会员权益

新手指南

应用程序菜单

```
shiyancelou:menu/ (master*) $ gcc -o init linktable.c menu.c test.c -m32 -static -
lpthread
shiyancelou:menu/ (master*) $ cp ../exec-lab/bin/hello-dynamic ./hello-dynamic
shiyancelou:menu/ (master*) $ mkdir -p lib lib32 [9:13:10]
shiyancelou:menu/ (master*) $ cp /lib/ld-linux.so.2 ./lib/ld-linux.so.2 [9:13:10]
shiyancelou:menu/ (master*) $ cp /lib32/libc.so.6 ./lib32/libc.so.6 [9:13:10]
shiyancelou:menu/ (master*) $ cp /lib32/libc.so.6 ./lib/libc.so.6 [9:13:11]
shiyancelou:menu/ (master*) $ cp -n ../rootfs.img ../rootfs.img.before-dynamic
shiyancelou:menu/ (master*) $ find init hello-static hello-dynamic lib lib32 | cpi
o -o -Hnewc | gzip -9 > ../rootfs.img
10388 块
shiyancelou:menu/ (master*) $ gzip -dc ../rootfs.img | cpio -it [9:13:34]
init
hello-static
hello-dynamic
lib
lib/ld-linux.so.2
lib/libc.so.6
lib32
lib32/libc.so.6
10388 块
shiyancelou:menu/ (master*) $ [9:13:36]
```



## 五、从系统调用到 `start_thread`

GDB 使用与 QEMU 内核对应的 `vmlinux` 设置 `sys_execve`、`load_elf_binary` 和 `start_thread` 断点。截图显示的主要调用路径为：

```

用户态 execv()
├── execve 系统调用
│   ├── SYSCALL_DEFINE3(execve) / Sys_execve
│   │   └── do_execve()
│   │       ├── exec_binprm() / search_binary_handler()
│   │       │   └── load_elf_binary()
│   │       │       ├── start_thread(regs, new_ip, new_sp)
│   │       │       └── 系统调用退出并返回用户态新映像

```

```

gdb vmlinux
Continuing.

Breakpoint 1, Sys_execve (filename=135049220, argv=-1075981244,
    envp=-1075979828) at fs/exec.c:1604
1604     SYSCALL_DEFINE3(execve,
(gdb) bt
#0 Sys_execve (filename=135049220, argv=-1075981244, envp=-1075979828)
    at fs/exec.c:1604
#1 <signal handler called>
#2 0xb77aec5c in ?? ()
#3 0x08049d47 in ?? ()
#4 0x0804a15a in ?? ()
#5 0x08052c6a in ?? ()
Backtrace stopped: previous frame inner to this frame (corrupt stack?)
(gdb) info args
filename = 135049220
argv = -1075981244
envp = -1075979828
(gdb) list
1599             old = ACCESS_ONCE(mm->flags);
1600             new = (old & ~MMF_DUMPABLE_MASK) | value;
1601             } while (cmpxchg(&mm->flags, old, new) != old);
1602     }
1603
1604     SYSCALL_DEFINE3(execve,
1605                     const char __user *, filename,
1606                     const char __user *const __user *, argv,
1607                     const char __user *const __user *, envp)
1608     {
(gdb) █

```

```

gdb vmlinux
(gdb) c
Continuing.

Breakpoint 2, load_elf_binary (bprm=0xc001ad00) at fs/binfmt_elf.c:572
572     {
(gdb) bt
#0 load_elf_binary (bprm=0xc001ad00) at fs/binfmt_elf.c:572
#1 0xc1130121 in search_binary_handler (bprm=0xc001ad00) at fs/exec.c:1374
#2 0xc1131191 in exec_binprm (bprm=<optimized out>) at fs/exec.c:1416
#3 do_execve_common (filename=0xc79d7000, argv=..., envp=...)
    at fs/exec.c:1513
#4 0xc11314bc in do_execve (__envp=<optimized out>, __argv=<optimized out>,
    filename=<optimized out>) at fs/exec.c:1555
#5 SYSC_execve (envp=<optimized out>, argv=<optimized out>,
    filename=<optimized out>) at fs/exec.c:1609
#6 Sys_execve (filename=135049220, argv=-1075981244, envp=-1075979828)
    at fs/exec.c:1604
#7 <signal handler called>
#8 0xb77aec5c in ?? ()
#9 0x08049d47 in ?? ()
#10 0x0804a15a in ?? ()
#11 0x08052c6a in ?? ()
Backtrace stopped: previous frame inner to this frame (corrupt stack?)
(gdb) info args
bprm = 0xc001ad00
(gdb) list
567         return PAGE_ALIGN(stack_top) - random_variable;
568     #endif
569     }
570
571     static int load_elf_binary(struct linux_binprm *bprm)
572     {
573         struct file *interpreter = NULL; /* to shut gcc up */
574         unsigned long load_addr = 0, load_bias = 0;
575         int load_addr_set = 0;
576         char * elf_interpreter = NULL;
(gdb) █

```

这条路径体现了内核的分工：`execve` 系统调用把文件名、参数和环境向量交给执行子系统；二进制格式处理器识别 ELF；`load_elf_binary` 检查 ELF 头和程序头、映射程序段、处理解释器并准备新地址空间；最后 `start_thread` 更新保存的用户态寄存器现场，让系统调用返回路径回到新程序的入口。

实验截图中，静态目标在 `start_thread` 处得到 `new_ip = 0x8048d0a`、`new_sp = 0xbfbf3080`。`new_ip` 与静态 ELF 的 `e_entry` 一致，说明无解释器时处理器从应用映像入口开始执行。`new_sp` 是传给 `start_thread` 的用户栈指针值；截图没有转储该地址附近的内存，因此不能把栈中每个字段说成现场直接读出。

```
(gdb) c
Continuing.

Breakpoint 3, start_thread (regs=0xc001ffb4, new_ip=134515978,
    new_sp=3216978048) at arch/x86/kernel/process_32.c:199
199      {
(gdb) bt
#0  start_thread (regs=0xc001ffb4, new_ip=134515978, new_sp=3216978048)
    at arch/x86/kernel/process_32.c:199
#1  0xc116cec0 in load_elf_binary (bprm=0xc001ad00) at fs/binfmt_elf.c:975
#2  0xc1130121 in search_binary_handler (bprm=0xc001ffb4) at fs/exec.c:1374
#3  0xc1131191 in exec_binprm (bprm=<optimized out>) at fs/exec.c:1416
#4  do_execve_common (filename=0xc79d7000, argv=..., envp=...)
    at fs/exec.c:1513
#5  0xc11314bc in do_execve (__envp=<optimized out>, __argv=<optimized out>,
    filename=<optimized out>) at fs/exec.c:1555
#6  SYSC_execve (envp=<optimized out>, argv=<optimized out>,
    filename=<optimized out>) at fs/exec.c:1609
#7  Sys_execve (filename=0, argv=0, envp=0) at fs/exec.c:1604
#8  <signal handler called>
#9  0xb77aec5c in ?? ()
(gdb) info args
regs = 0xc001ffb4
new_ip = 134515978
new_sp = 3216978048
(gdb) p/x new_ip
$1 = 0x8048d0a
(gdb) p/x new_sp
$2 = 0xbfbf3080
(gdb) █
```

## 六、动态 ELF 为什么先进入解释器

动态 ELF 的 `PT_INTERP` 指定动态链接器。实验确认解释器路径为 `/lib/ld-linux.so.2`，程序依赖 `libc.so.6`。Linux 3.18.6 的 ELF 装载路径会同时安排主程序和解释器映像；当存在解释器时，`start_thread` 的 `new_ip` 指向解释器入口，而不是直接等于应用程序的 `e_entry`。

本次 GDB 截图中，动态 ELF 的 `new_ip = 0xb77d90d0`，`new_sp = 0xbfed8be0`。应用本身的 `e_entry` 是 `0x8048350`，两者不同；结合 `PT_INTERP`，可以解释为处理器先进入动态链接器。动态链接器随后完成共享对象映射和必要的重定位，再把控制权交给应用程序入口。因而，`execve` 返回用户态时，静态程序通常已在自身入口开始运行；动态程序则先从 ELF 解释器入口运行，之后才进入应用入口。

```

shiyanolou:~/ $ cd ~/LinuxKernel/exec-lab [9:10:34]
shiyanolou:exec-lab/ $ ldd bin/hello-dynamic [9:10:41]
        linux-gate.so.1 => (0xf7f0f000)
        libc.so.6 => /lib32/libc.so.6 (0xf7d4d000)
        /lib/ld-linux.so.2 (0xf7f11000)
shiyanolou:exec-lab/ $ readelf -l bin/hello-dynamic | grep -A1 INTERP [9:10:41]
    INTERP               0x0000154 0x08048154 0x08048154 0x000013 0x000013 R    0x1
    [正在请求程序解释器：/lib/ld-linux.so.2]
shiyanolou:exec-lab/ $ readelf -d bin/hello-dynamic | grep NEEDED [9:10:41]
    0x00000001 (NEEDED)           共享库：[libc.so.6]
shiyanolou:exec-lab/ $ [9:10:42]

```

```

gdb vmlinux
Continuing.

Breakpoint 3, start_thread (regs=0xc7bf9fb4, new_ip=3078459600, new_sp=3220016096)
    at arch/x86/kernel/process_32.c:199
199      {
(gdb) bt
#0  start_thread (regs=0xc7bf9fb4, new_ip=3078459600, new_sp=3220016096)
    at arch/x86/kernel/process_32.c:199
#1  0xc116cec0 in load_elf_binary (bprm=0xc7bc8d00) at fs/binfmt_elf.c:975
#2  0xc1130121 in search_binary_handler (bprm=0xc7bf9fb4) at fs/exec.c:1374
#3  0xc1131191 in exec_binprm (bprm=<optimized out>) at fs/exec.c:1416
#4  do_execve_common (filename=0xc7a0a000, argv=..., envp=...) at fs/exec.c:1513
#5  0xc11314bc in do_execve (__envp=<optimized out>, __argv=<optimized out>,
    filename=<optimized out>) at fs/exec.c:1555
#6  SYSC_execve (envp=<optimized out>, argv=<optimized out>, filename=<optimized out>)
    at fs/exec.c:1609
#7  Sys_execve (filename=0, argv=0, envp=0) at fs/exec.c:1604
#8  <signal handler called>
#9  0xb775cc5c in ?? ()
(gdb) info args
regs = 0xc7bf9fb4
new_ip = 3078459600
new_sp = 3220016096
(gdb) list
194          return err;
195      }
196
197      void
198      start_thread(struct pt_regs *regs, unsigned long new_ip, unsigned long new_sp)
199      {
200          set_user_gs(regs, 0);
201          regs->fs          = 0;
202          regs->ds          = __USER_DS;
203          regs->es          = __USER_DS;
(gdb) p/x new_ip
$1 = 0xb77d90d0
(gdb) p/x new_sp
$2 = 0xbfed8be0
(gdb)

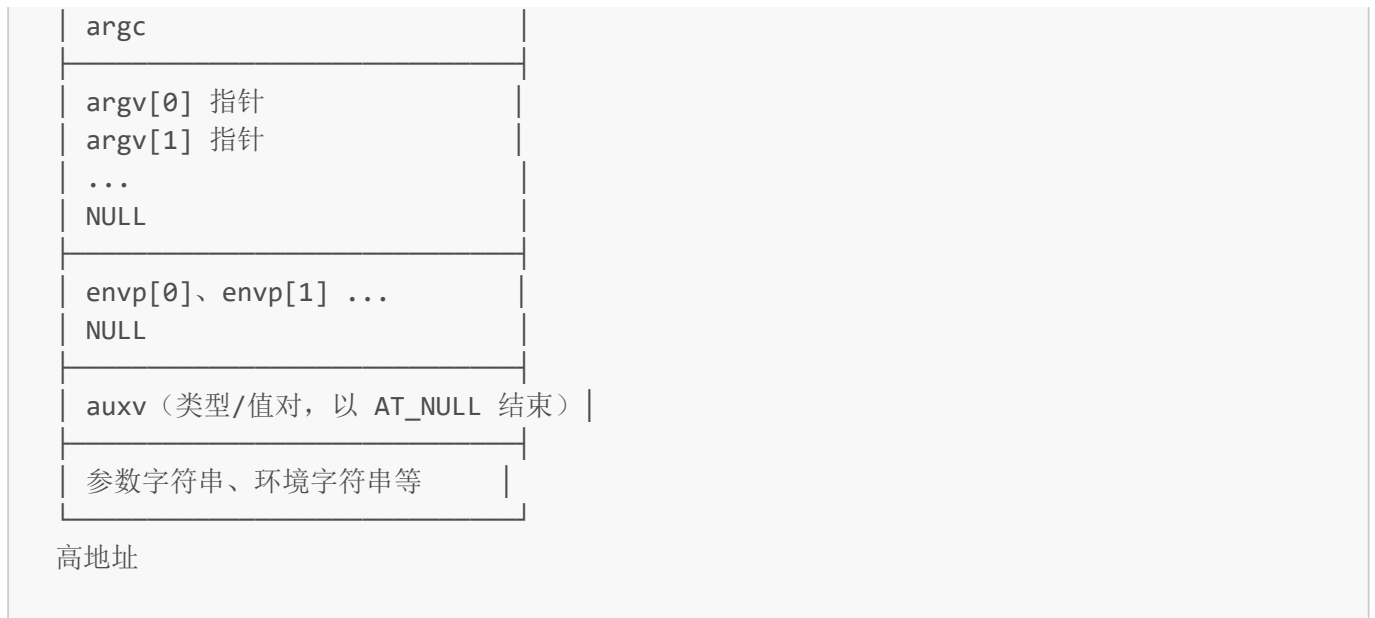
```

动态程序的入口与栈地址和静态程序不同，也可能受地址空间布局随机化及映射布局影响。这里记录的是本次截图中的具体数值，不应当视为每次运行都固定的地址。

## 七、初始用户栈承载什么

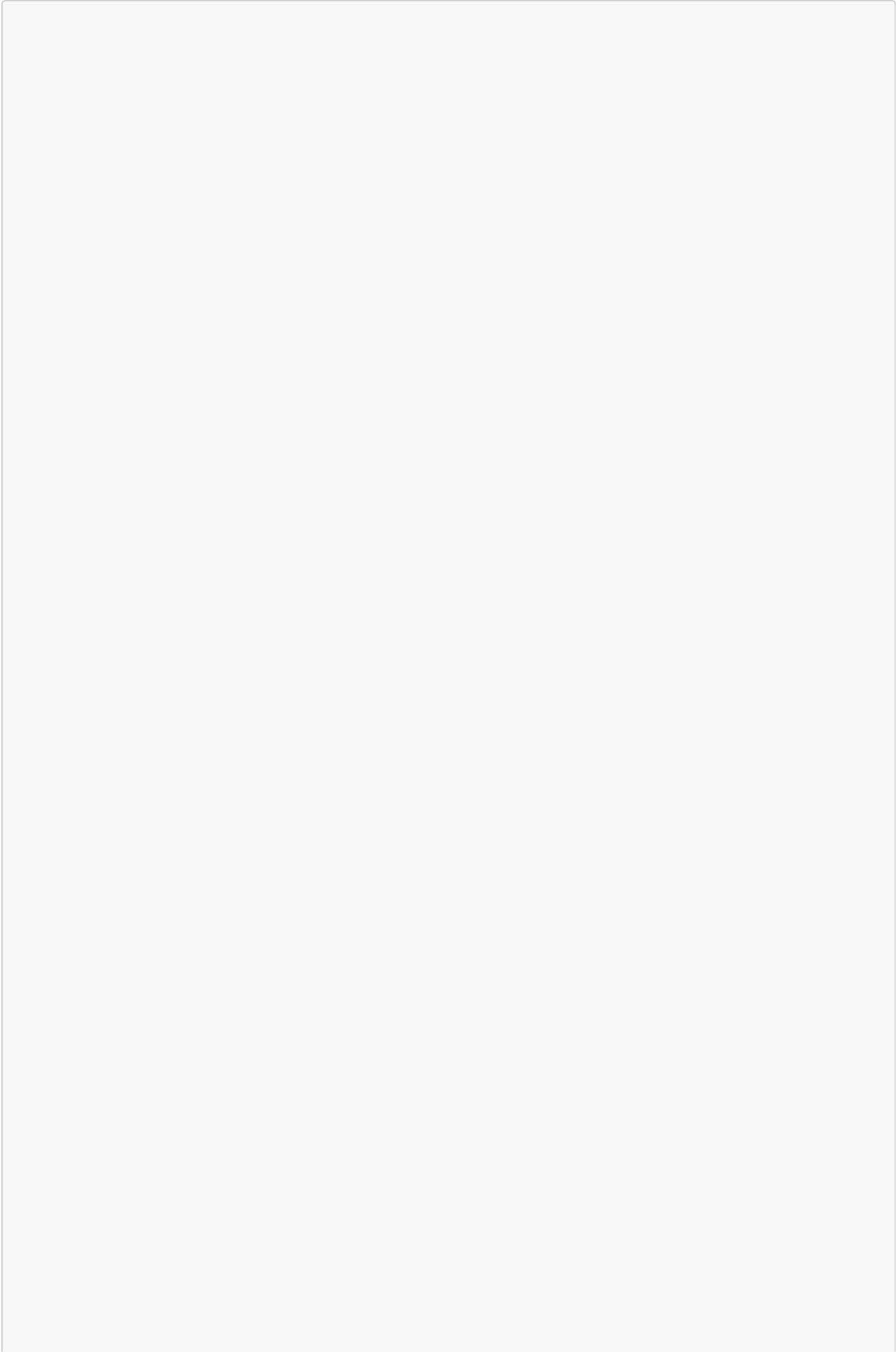
无论静态还是动态程序，内核都要为新映像准备参数、环境变量和辅助向量等初始信息。对 32 位 x86 用户态，初始栈顶可概念化为：

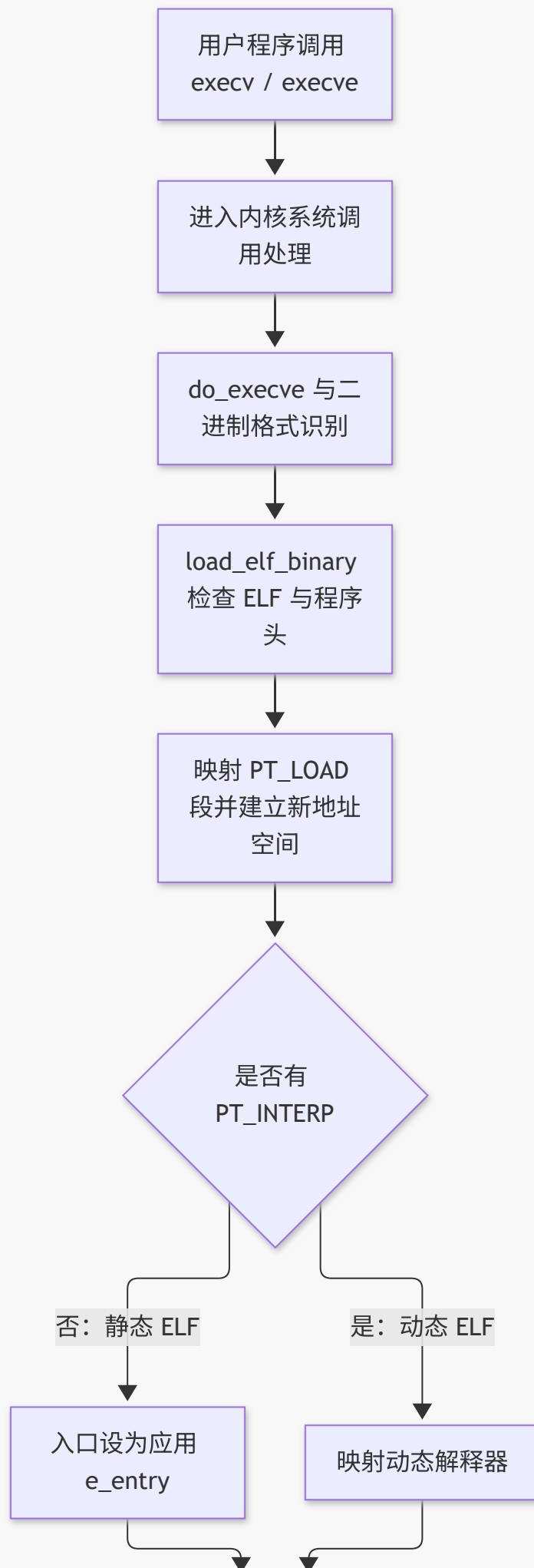
低地址 / 初始 ESP 指向

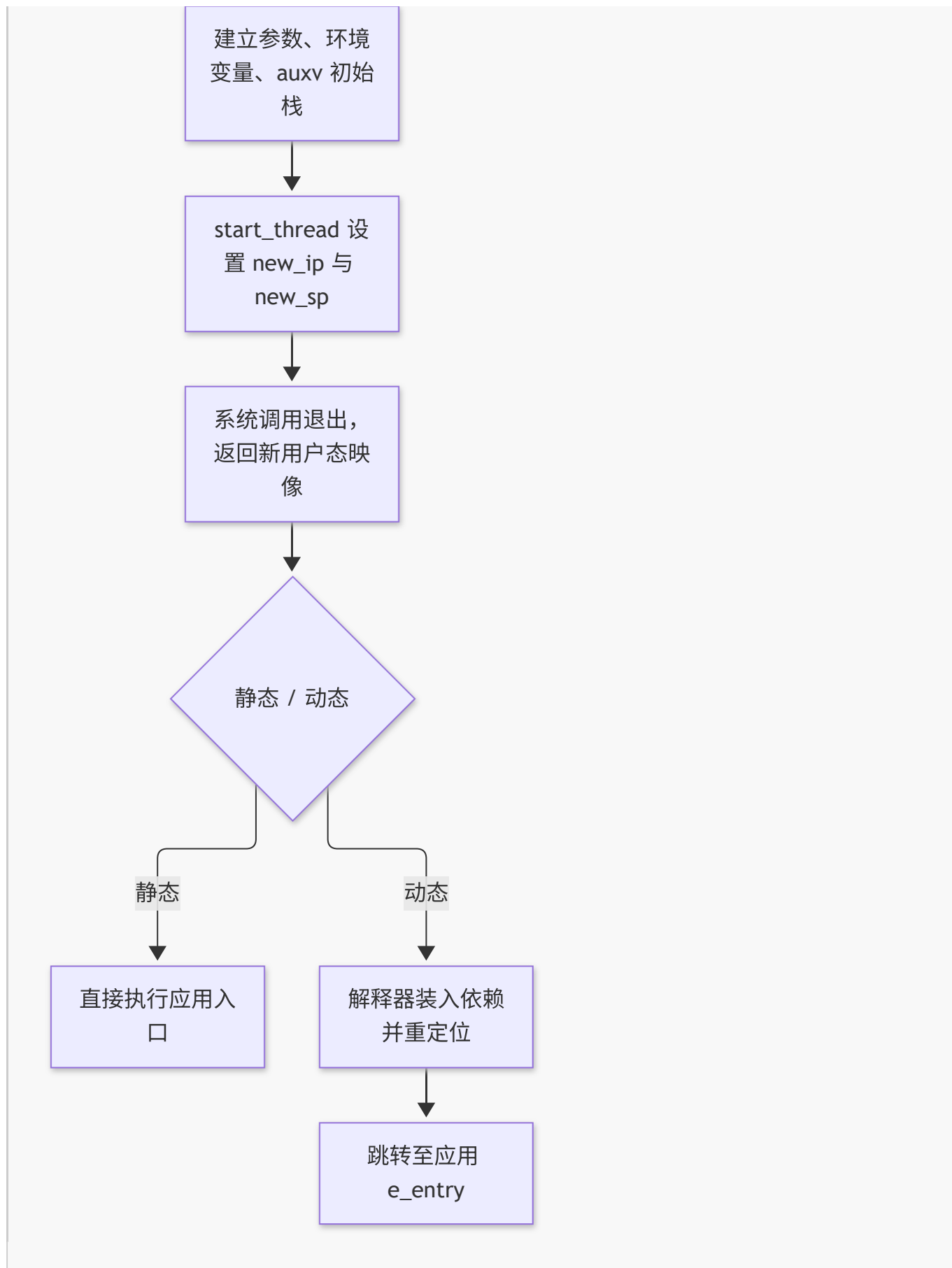


这表示程序无需沿用旧程序的调用栈帧；它从内核建立的全新启动栈和入口寄存器现场开始。程序运行时的 C 运行库启动代码再根据 `argc`、`argv`、`envp` 和辅助向量完成初始化并调用 `main`。此图是根据 ELF 用户态启动约定和 Linux 装载机制整理的概念图，本次实验只观察了 `new_sp` 数值和目标程序打印出的参数，没有用 GDB 逐字节检查栈内存。

## 八、装载与启动流程







## 九、总结：计算机怎样开始执行一个新程序

这次实验让我看到，“执行一个程序”不是简单地把文件读进内存再跳过去。用户态先通过系统调用提出请求；内核识别 ELF 格式，验证文件并按程序头建立映射，准备新的地址空间、入口寄存器和初始栈；随后处理器从 `start_thread` 设置的用户态入口继续执行。系统调用边界连接了受保护的内核机制和用户程序的运行现场。

静态程序的入口直接来自 ELF 的 `e_entry`；动态程序还要经过 `PT_INTERP` 指定的动态链接器，由解释器准备共享库和重定位后再进入应用入口。`argv`、环境变量和辅助向量则通过初始用户栈传给启动代码。由此我理解，计算机“开始工作”依赖的不只是指令地址，还包括地址空间、寄存器现场、栈中约定的数据以及内核与用户态之间清晰的交接。

## 参考与版权说明

- Linux 3.18.6 内核源码：`fs/exec.c`、`fs/binfmt_elf.c`、`arch/x86/kernel/process_32.c`。
- System V ABI：ELF 文件格式与进程初始化约定。

叶丁再（学号：20262809）。原创作品转载请注明出处：《Linux 内核分析》MOOC 课程：

<http://mooc.study.163.com/course/USTC-100002900>。

本文为博客草稿；发布后请将实际博客 URL 提交到课程平台，不要把本地文件路径当作发布链接。